

# การเก็บข้อมูลแบบถาวรด้วย **Shared Storage** ใน **Flutter**

By Prasertsak U., Ph.D

# Outlines

- Shared Preferences
- Secure Storage

# Shared Preferences คืออะไร?

- **Shared Preferences** คือกลไกการเก็บข้อมูลแบบ Key-Value ที่มีขนาดเล็กและใช้งานง่าย ซึ่งเป็นวิธีที่เหมาะสมสำหรับการเก็บข้อมูลเล็กๆ น้อยๆ เช่น:
  - การตั้งค่าของผู้ใช้ (User settings) เช่น Theme ของแอป, ขนาดตัวอักษร
  - สถานะการล็อกอิน (Login status)
  - ข้อมูลผู้ใช้ล่าสุด (Last user logged in)
  - สถานะของ Tutorial ระหว่างที่ผู้ใช้กำลังรับชม เป็นต้น

# หลักการทำงาน

- **Key-Value:** ข้อมูลจะถูกเก็บในรูปแบบของคู่ Key (String) และ Value (ประเภทข้อมูลพื้นฐาน เช่น int, double, bool, String, หรือ List<String>)
- **Persistent:** ข้อมูลจะยังคงอยู่แม้จะปิดแอปไปแล้ว สามารถอ่านข้อมูลเดิมมาใช้งานได้เมื่อเข้า App ครั้งถัดไป
- **Simple:** การใช้งานง่าย ไม่ซับซ้อนเหมือนฐานข้อมูล (Database)
- **Platform-specific:** Shared Preferences จะใช้กลไกการเก็บข้อมูลพื้นฐานของแต่ละระบบปฏิบัติการ
  - **Android:** SharedPreferences
  - **Web:** localStorage
  - **iOS:** NSUserDefaults

# ข้อจำกัด

- ไม่เหมาะสำหรับข้อมูลขนาดใหญ่: Shared Preferences  
ถูกออกแบบมาเพื่อเก็บข้อมูลขนาดเล็กเท่านั้น หากต้องการเก็บข้อมูลจำนวนมาก  
หรือมีโครงสร้างซับซ้อน ควรพิจารณาใช้ฐานข้อมูล เช่น SQLite หรือ Hive
- ไม่มีการเข้ารหัส (Encryption): โดย Default แล้วข้อมูลที่เก็บไว้จะไม่ได้ถูกเข้ารหัส  
หากต้องการเก็บข้อมูลที่ละเอียดอ่อน ควรใช้ Plugin ที่มีการเข้ารหัสเพิ่มเติม

# Using Shared storage package

- Add a dependency

No guarantee that writes will be persisted to disk after returning, so this plugin must not be used for storing critical data

```
dependencies:  
  shared_preferences: ^2.0.12
```

Alternatively, your editor might support `flutter pub get`. Check the docs for your editor to learn more.

Import it

Now in your Dart code, you can use:

```
import 'package:shared_preferences/shared_preferences.dart';
```

# Store Key-Value data on shared storage

- Save Data

To persist data, use the setter methods provided by the `SharedPreferences` class. Setter methods are available for various primitive types, such as `setInt`, `setBool`, and `setString`.

Setter methods do two things: First, synchronously update the key-value pair in-memory. Then, persist the data to disk.

```
// obtain shared preferences
final prefs = await SharedPreferences.getInstance();

// set value
prefs.setInt('counter', counter);
```



# Store Key-Value data on shared storage (cont.)

- Read Data & Remove Data

To read data, use the appropriate getter method provided by the `SharedPreferences` class. For each setter there is a corresponding getter. For example, you can use the `getInt`, `getBool`, and `getString` methods.

```
final prefs = await SharedPreferences.getInstance();  
  
// Try reading data from the counter key. If it doesn't exist, return 0.  
final counter = prefs.getInt('counter') ?? 0;
```



To delete data, use the `remove()` method.

```
final prefs = await SharedPreferences.getInstance();  
  
prefs.remove('counter');
```



# async, await และ Future<T>

- **async** และ **await** จะใช้คู่กัน คือการจัดการกับ "เวลา"
  - **async** (Asynchronous): ใส่ไว้ที่ฟังก์ชันเพื่อบอกว่า "ฟังก์ชันนี้มีการทำงานที่ต้องรอ" และจะคืนค่ากลับไปเป็น Future เสมอ
  - **await** (Wait): ใส่ไว้หน้าคำสั่งที่ต้องใช้เวลา (เช่น การดึงข้อมูลจาก API) เพื่อบอกโปรแกรมว่า "ให้หยุดตรงนี้นะจนกว่าจะได้ข้อมูลกลับมาหรือทำงานเสร็จค่อยไปบรรทัดถัดไป"
- **Future<T>** คือ "สัญญา" ว่าเราจะได้ข้อมูลประเภท T ในอนาคต (**T** คือชนิดของข้อมูลที่จะต้องระบุ เช่น double, int, bool หรือ object ของ Model Class ที่สร้างขึ้น และอื่น ๆ เป็นต้น)
- สำหรับผู้ที่เรียกใช้ฟังก์ชันที่มีการส่งค่ากลับเป็น Future<T> ก็มักจะต้องระบุ await ไว้ด้านหน้าด้วย เพื่อรอการส่งค่ากลับ เช่นกัน และเมื่อมีการใช้ await ที่ใดที่ชื่อของฟังก์ชันนั้นก็ต้องระบุ async ไว้ด้วยเสมอ

# เรียนรู้การทำงานจากตัวอย่าง

- สร้าง Flutter Project ชื่อว่า `shared_storage_demo`
- นำโปรแกรมจาก `/lib` ที่ได้จาก โปรแกรมตัวอย่าง มาวางแทนที่ `lib` เดิม

เข้าสู่ระบบ

Login

ชื่อผู้ใช้  
admin

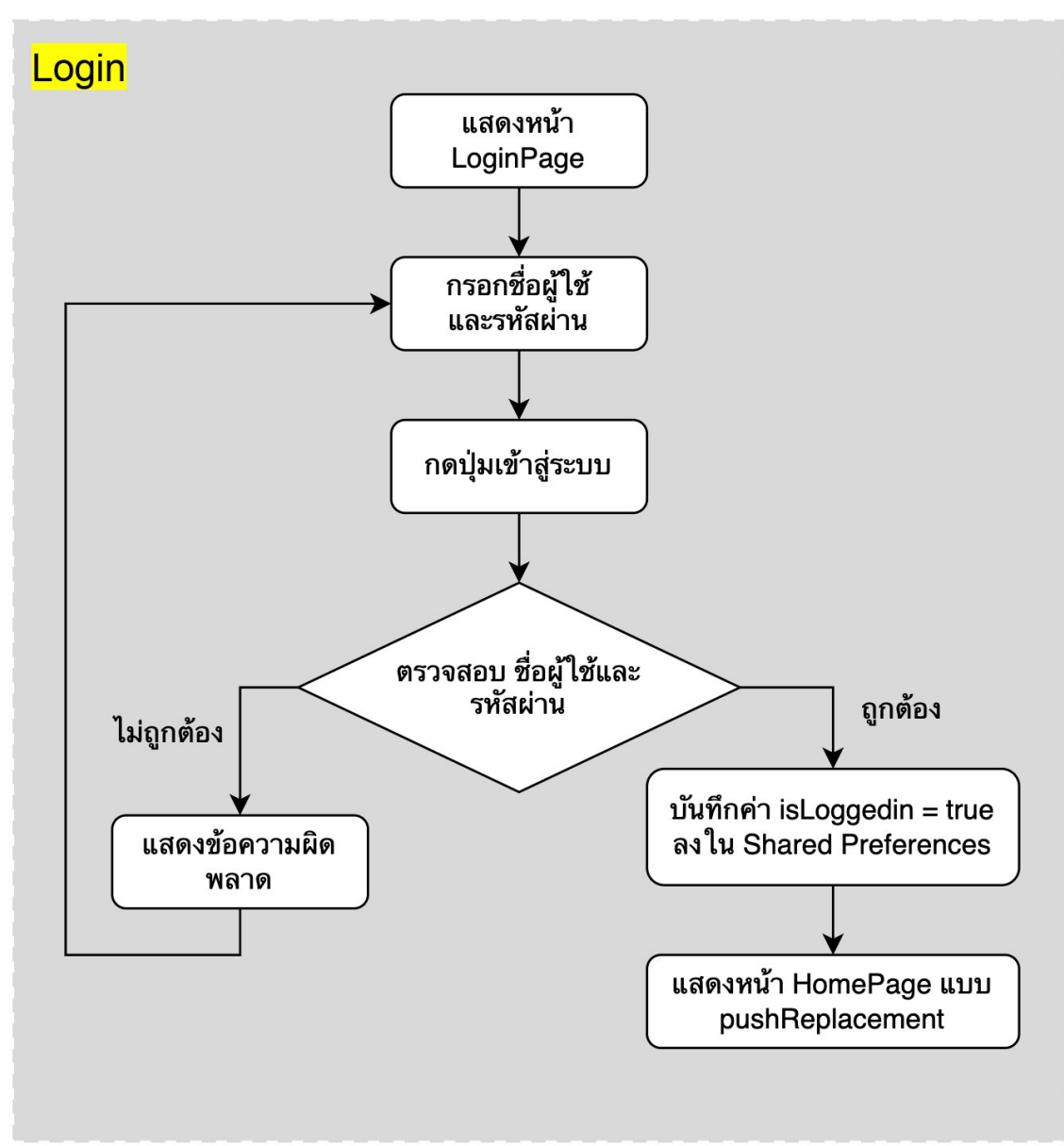
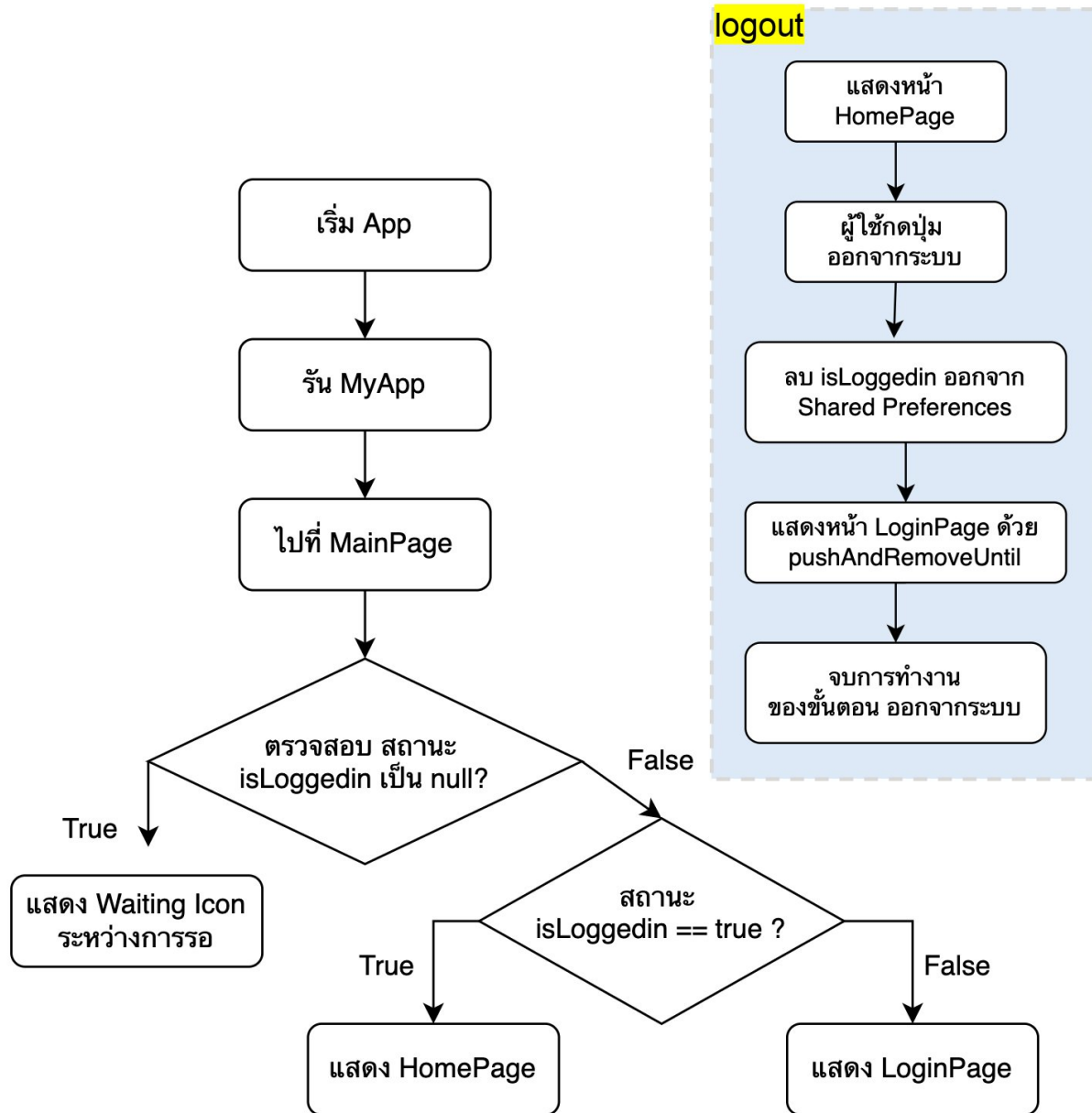
รหัสผ่าน  
#####

เข้าสู่ระบบ

หน้าหลัก

Welcome to Home Page!

ออกจากระบบ



# ตัวอย่างการใช้งาน Shared Preferences

- เมื่อเข้าสู่ App จะเชื่อมไปยังหน้า MainPage จากนั้นจะเริ่มตรวจสอบสถานะการ Login ของผู้ใช้งานที่ที่ App เปิดใช้งาน โดยใช้การทำงานของ SharedPreferences เพื่ออ่านค่าเดิมที่เก็บไว้ มาตรวจสอบ

```
1 import 'package:flutter/material.dart';
2 import 'package:shared_preferences/shared_preferences.dart';
3
4 Run | Debug | Profile
5 void main() {
6   runApp(const MyApp());
7 }
8
9 // MyApp is the root of the application.
10 class MyApp extends StatelessWidget {
11   const MyApp({super.key});
12
13   @override
14   Widget build(BuildContext context) {
15     return MaterialApp(
16       debugShowCheckedModeBanner: false,
17       title: 'Login App',
18       theme: ThemeData(
19         colorScheme: ColorScheme.fromSeed(seedColor: Colors.blue),
20         useMaterial3: true,
21       ), // ThemeData
22       home: const MainPage(),
23     ); // MaterialApp
24 }
```

# ตัวอย่างการใช้งาน Shared Preferences

- การตรวจสอบสถานะการ Login ของผู้ใช้ทันทีที่ App เปิดใช้งาน โดยใช้ SharedPreferences เพื่ออ่านค่าที่เก็บไว้

```
54  @override
55  Widget build(BuildContext context) {
56    // Show a circular progress indicator while checking the login
57    if (_isLoggedIn == null) {
58      return const Scaffold(
59        body: Center(
60          child: CircularProgressIndicator(),
61        ), // Center
62      ); // Scaffold
63    }
64    // If logged in, navigate to HomePage; otherwise, navigate to
65    return _isLoggedIn! ? const HomePage() : const LoginPage();
66  }
67 }
```

```
26  // MainPage is responsible for checking the login status and navigating
27  // to the appropriate screen (LoginPage or HomePage).
28  class MainPage extends StatefulWidget {
29    const MainPage({super.key});
30
31    @override
32    State<MainPage> createState() => _MainPageState();
33  }
34
35  class _MainPageState extends State<MainPage> {
36    // A boolean to hold the login status. Initially null to show a loading indicator.
37    bool? _isLoggedIn;
38
39    @override
40    void initState() {
41      super.initState();
42      _checkLoginStatus();
43    }
44
45    // Asynchronous function to check if the user is logged in using SharedPreferences.
46    Future<void> _checkLoginStatus() async {
47      final prefs = await SharedPreferences.getInstance();
48      final isLoggedIn = prefs.getBool('isLoggedIn') ?? false;
49      setState(() {
50        _isLoggedIn = isLoggedIn;
51      });
52    }
53  }
```

Edit #1



# ตัวอย่างการใช้งาน Shared Preferences

- การทำงานในส่วนของการ Login ในตัวอย่างนี้จะใช้การกำหนดชื่อและรหัสผู้ใช้แบบเจาะจงเพื่อความง่ายในการทดสอบ

| Storage         |                        | Key                | Value |
|-----------------|------------------------|--------------------|-------|
| ▼ Local storage |                        |                    |       |
|                 | http://localhost:53215 | flutter.isLoggedIn | true  |

## pushReplacement:

เป็นการแทนที่หน้าจอปัจจุบันด้วยหน้าจอใหม่ โดยหน้าจอปัจจุบันจะถูกนำออกจาก Stack ทำให้ผู้ใช้ไม่สามารถกดปุ่มย้อนกลับเพื่อกลับมาที่หน้าจอเดิมได้

จากตัวอย่าง: เมื่อผู้ใช้ล็อกอินสำเร็จ แล้วเปลี่ยนจากหน้า LoginPage ไปหน้า HomePage ควรใช้ pushReplacement เพื่อป้องกันไม่ให้ผู้ใช้กดปุ่มย้อนกลับไปที่หน้าล็อกอินได้อีก

```
77 class _LoginPageState extends State<LoginPage> {
78   final _usernameController = TextEditingController();
79   final _passwordController = TextEditingController();
80
81   // Hardcoded account credentials for demonstration.
82   static const String _hardcodedUsername = 'admin';
83   static const String _hardcodedPassword = '123456';
84
85   // Function to handle the login logic.
86   Future<void> _login() async {
87     // Check if the entered credentials match the hardcoded ones.
88     if (_usernameController.text == _hardcodedUsername &&
89       _passwordController.text == _hardcodedPassword) {
90       // If successful, save the login status to SharedPreferences.
91       final prefs = await SharedPreferences.getInstance();
92       await prefs.setBool('isLoggedIn', true);
93     }
94     // Navigate to the HomePage and replace the current route.
95     // This prevents the user from going back to the login page with the back button.
96     if (mounted) {
97       Navigator.of(context).pushReplacement(
98         MaterialPageRoute(builder: (context) => const HomePage()),
99       );
100   }
101   } else {
102     // Show an error message if credentials are incorrect.
103     if (mounted) {
104       ScaffoldMessenger.of(context).showSnackBar(
105         const SnackBar(content: Text('ชื่อผู้ใช้หรือรหัสผ่านไม่ถูกต้อง')),
106       );
107     }
108   }
109 }
```

Edit #2

# ตัวอย่างการใช้งาน Shared Preferences

- `mounted` คือคุณสมบัติ (Property) ของ State Object ใน StatefulWidget ซึ่งเป็นค่า Boolean ที่จะบอกสถานะว่า State Object นั้นยังคงอยู่ใน Widget Tree หรือไม่
- `mounted = true`: Widget นี้ยังคงอยู่ใน Widget Tree และพร้อมใช้งาน
- `mounted = false`: Widget นี้ถูกถอดออกจาก Widget Tree แล้วหรือก็คือถูก Disposed ไปแล้ว

```
77 class _LoginPageState extends State<LoginPage> {
78   final _usernameController = TextEditingController();
79   final _passwordController = TextEditingController();
80
81   // Hardcoded account credentials for demonstration.
82   static const String _hardcodedUsername = 'admin';
83   static const String _hardcodedPassword = '123456';
84
85   // Function to handle the login logic.
86   Future<void> _login() async {
87     // Check if the entered credentials match the hardcoded ones.
88     if (_usernameController.text == _hardcodedUsername &&
89       _passwordController.text == _hardcodedPassword) {
90       // If successful, save the login status to SharedPreferences.
91       final prefs = await SharedPreferences.getInstance();
92       await prefs.setBool('isLoggedIn', true);
93
94       // Navigate to the HomePage and replace the current route.
95       // This prevents the user from going back to the login page with the back button.
96       if (mounted) {
97         Navigator.of(context).pushReplacement(
98           MaterialPageRoute(builder: (context) => const HomePage()),
99         );
100       }
101     } else {
102       // Show an error message if credentials are incorrect.
103       if (mounted) {
104         ScaffoldMessenger.of(context).showSnackBar(
105           const SnackBar(content: Text('ชื่อผู้ใช้หรือรหัสผ่านไม่ถูกต้อง')),
106         );
107       }
108     }
109   }
110 }
```

# ทำไมต้องใช้ if (mounted)? (ก่อนการแสดงผล Snackbar)

ปัญหาหลักที่ if (mounted) เข้ามาช่วยแก้คือเมื่อเรามีโค้ดที่ทำงานแบบ Asynchronous เช่น การเรียก API, การอ่านข้อมูลจาก SharedPreferences หรือการดาวน์โหลดไฟล์ หากเกิดสถานการณ์เหล่านี้ เช่น:

- เมื่ออยู่ในหน้าจอ A และเรียกใช้งานฟังก์ชัน Asynchronous (เช่น `_login()` ที่มีการเรียกใช้ SharedPreferences หรือทำงานเป็นเวลานาน) และก่อนที่ฟังก์ชันจะทำงานสำเร็จ ผู้ใช้กดปุ่มย้อนกลับหรือเปลี่ยนไปหน้าจออื่น ในกรณีนี้หน้าจอ A จะถูกทำลาย (disposed) ไปจาก Widget Tree
- ต่อมาเมื่อฟังก์ชัน Asynchronous ทำงานเสร็จสิ้น ก็จะพยายามที่จะเรียกใช้ `setState()` หรือแสดง Snackbar เพื่ออัปเดต UI ของหน้าจอ A แต่เนื่องจากหน้าจอ A ไม่ได้อยู่ใน Widget Tree แล้ว การพยายามอัปเดต UI จะทำให้เกิดข้อผิดพลาดและแอปอาจจะเกิดการ **Crash** ได้
- ดังนั้นการใช้ if (mounted) จึงเป็นการช่วยตรวจสอบก่อนว่า "หน้าจอที่กำลังจะอัปเดตนี้ยังอยู่ไหม?" ถ้า true ก็ให้ทำงานต่อ แต่ถ้า false ก็จะไม่ทำอะไรเลย ซึ่งเป็นการป้องกันข้อผิดพลาดได้



# ตัวอย่างการใช้งาน Shared Preferences

- **pushAndRemoveUntil:**  
เป็นคำสั่งเปิดหน้าจอโดยจะเพิ่มหน้าจอใหม่  
เข้าไปใน Stack แล้วลบหน้าจออื่นๆ ทั้งหมด
- **จากตัวอย่าง:** เมื่อผู้ใช้กด "ออกจากระบบ"  
(Logout) จากหน้า HomePage  
คุณต้องการให้กลับไป LoginPage  
และล้างหน้าจออื่นๆ ที่เคยดูมาก่อนหน้า  
ทั้งหมดออกไปจาก Stack เพื่อไม่ให้ผู้ใช้  
ย้อนกลับมาดูได้อีก

```
162 // HomePage is the screen shown after a successful login.
163 class HomePage extends StatelessWidget {
164   const HomePage({super.key});
165
166   // Function to handle the logout logic.
167   Future<void> _logout(BuildContext context) async {
168     // Remove the 'isLoggedIn' key from SharedPreferences.
169     final prefs = await SharedPreferences.getInstance();
170     await prefs.remove('isLoggedIn');
171
172     // Navigate to the LoginPage and replace all previous routes.
173     // This ensures the user cannot use the back button to get back to the home page.
174     if (context.mounted) {
175       Navigator.of(context).pushAndRemoveUntil(
176         MaterialPageRoute(builder: (context) => const LoginPage()),
177         (Route<dynamic> route) => false,
178       );
179     }
180   }
}
```

Edit #3

(Route<dynamic> route) => false หมายความว่า "สำหรับทุกๆ หน้าจอที่พบ ให้คืนค่า false เสมอ" ซึ่งส่งผลให้แอปพลิเคชันจะลบทุกหน้าจอที่มีอยู่ใน Stack ออกทั้งหมดจนกว่าจะไม่มีหน้าจอเหลืออยู่เลย จากนั้นจึงจะเพิ่มหน้าจอใหม่ (LoginPage) เข้าไปแทนที่

\* หากค่า route => true หน้าจอ นั้นจะไม่ถูกลบออกจาก stack

# What is Secure Storage?

- ใช้สำหรับเก็บข้อมูลสำคัญแบบ เข้ารหัส บนอุปกรณ์
- บน Android: ใช้ Keystore + AES/RSA
- บน iOS: ใช้ Keychain

With Flutter:

```
$ flutter pub add flutter_secure_storage
```

This will add a line like this to your package's pubspec.yaml (and run an implicit `flutter pub get`):

```
dependencies:  
  flutter_secure_storage: ^9.2.4
```

# รูปแบบการบันทึก อ่าน และลบ Secure Storage

```
import 'package:flutter_secure_storage/flutter_secure_storage.dart';  
  
final secureStorage = FlutterSecureStorage();
```

- การบันทึกข้อมูล

```
await secureStorage.write(key: 'auth_token', value: 'abc123xyz');
```

- การอ่านข้อมูล

```
String? token = await secureStorage.read(key: 'auth_token');
```

- การลบข้อมูล

```
await secureStorage.delete(key: 'auth_token');
```

# ตัวอย่างการใช้งาน Secure Storage

- ตัวอย่างการเรียกใช้งาน Secure Storage และการอ่านข้อมูลด้วยการระบุ key name ตามที่ได้บันทึกไว้

```
1 import 'package:flutter/material.dart';
2 import 'package:flutter_secure_storage/flutter_secure_storage.dart';

31 class MainPage extends StatefulWidget {
32   const MainPage({super.key});
33
34   @override
35   State<MainPage> createState() => _MainPageState();
36 }
37
38 class _MainPageState extends State<MainPage> {
39   // A boolean to hold the login status. Initially null to show a loading indicator.
40   bool? _isLoggedIn;
41
42   @override
43   void initState() {
44     super.initState();
45     _checkLoginStatus();
46   }
47
48   // Asynchronous function to check if the user is logged in using SharedPreferences.
49   Future<void> _checkLoginStatus() async {
50     final secureStorage = const FlutterSecureStorage();
51     final loggedIn = await secureStorage.read(key: 'isLoggedIn') == 'true';
52     // Update the state with the login status.
53     setState(() {
54       _isLoggedIn = loggedIn;
55     });
56   }
```

# ตัวอย่างการใช้งาน Secure Storage

- ตัวอย่างการบันทึกข้อมูลลง

Secure Storage

```
1 import 'package:flutter/material.dart';
2 import 'package:flutter_secure_storage/flutter_secure_storage.dart';
```

```
22 // Function to handle the login logic.
23 Future<void> _login() async {
24   // Check if the entered credentials match the hardcoded ones.
25   if (_usernameController.text == _hardcodedUsername &&
26       _passwordController.text == _hardcodedPassword) {
27     // If successful, save the login status to Shared Preferences.
28     final secureStorage = const FlutterSecureStorage();
29     await secureStorage.write(key: 'isLoggedIn', value: 'true');
30
31     // Navigate to the HomePage and replace the current route.
32     // This prevents the user from going back to the login page with the back button.
33     if (mounted) {
34       Navigator.of(context).pushReplacement(
35         MaterialPageRoute(builder: (context) => const HomePage()),
36       );
37     }
38   } else {
39     // Show an error message if credentials are incorrect.
40     if (mounted) {
41       ScaffoldMessenger.of(context).showSnackBar(
42         const SnackBar(content: Text('ชื่อผู้ใช้หรือรหัสผ่านไม่ถูกต้อง')),
43       );
44     }
45   }
46 }
```

| Key                             | Value                                         |
|---------------------------------|-----------------------------------------------|
| FlutterSecureStorage            | gTBdCKYqTSSXN4LICyWJMbJ/EnfArizsj4k9fCyPLE=   |
| FlutterSecureStorage.isLoggedIn | PcqsqnkKoznSz+0f.9h65+voURBKduzzErijcT09ZZWg= |



# ตัวอย่างการใช้งาน Secure Storage

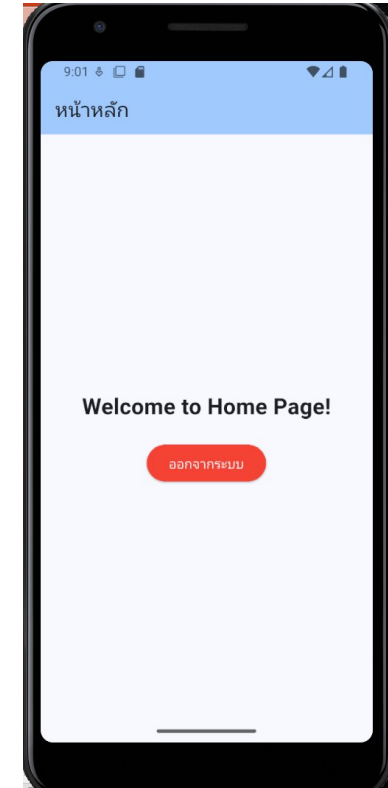
- ตัวอย่างการลบข้อมูลออกจาก Secure Storage

```
1 import 'package:flutter/material.dart';  
2 import 'package:flutter_secure_storage/flutter_secure_storage.dart';
```

```
10 // Function to handle the logout logic.  
11 Future<void> _logout(BuildContext context) async {  
12   // Remove the 'isLoggedIn' key from SharedPreferences.  
13   final secureStorage = const FlutterSecureStorage();  
14   await secureStorage.delete(key: 'isLoggedIn');  
15  
16   // Navigate to the LoginPage and replace all previous routes.  
17   // This ensures the user cannot use the back button to get back to the home page.  
18   if (context.mounted) {  
19     Navigator.of(context).pushAndRemoveUntil(  
20       MaterialPageRoute(builder: (context) => const LoginPage()),  
21       (Route<dynamic> route) => false,  
22     );  
23   }  
24 }
```

# วิธีการทดสอบ

- การทดสอบบน **Android Emulator**
  - หลังจาก Login เข้าสู่หน้า Home เรียบร้อยแล้ว ให้ออกจาก App ทันทีโดยไม่ต้องกดปุ่ม ออกจากระบบ (เพื่อต้องการให้ข้อมูลค้างอยู่ใน shared storage )
  - จากนั้นให้เข้า App อีกครั้ง ผลที่ได้หลังจากนั้นควรจะต้องเข้าสู่หน้า Home ได้ทันทีโดยไม่ต้องผ่านการ Login เนื่องจากสถานะการ login ล่าสุดยังคงค้างอยู่ในที่จัดเก็บ



**Remark:** ตัวอย่างเพื่อการทดสอบเท่านั้น กรณีจริงการปิดออกโดยไม่ได้ Logout ก็ควรลบสถานะการ Login ออกจาก SharedPreferences ด้วยเสมอ



# วิธีการทดสอบ (cont.)

- การทดสอบบน **Chrome Browser**

- เนื่องจากการรันบน Chrome จะมีการสุ่มหมายเลข Port ใหม่ทุกครั้ง ทำให้พื้นที่ใน Local Storage ของ Chrome เปลี่ยนไปเรื่อย ๆ ตามหมายเลข Port ที่เปลี่ยนไป จึงเป็นเหตุให้ไม่สามารถเข้าถึงข้อมูลเดิมที่เก็บไว้ได้
- รวมทั้งทุกครั้งที่เราสั่ง flutter run (หรือกดปิดหน้าต่าง Chrome ไป แล้วสั่งรันซ้ำ) Flutter จะลบหรือสร้างโฟลเดอร์ชั่วคราวนี้ ขึ้นมาใหม่เสมอ ส่งผลให้ localStorage, Cookies, และ Session ที่เคยฝังอยู่ถูกทำลายทิ้งไปพร้อมกับ Profile ชั่วคราวนั้นทันที จึงไม่เหลือข้อมูลเดิมที่เก็บไว้
- วิธีการทดสอบ จึงต้องกำหนดหมายเลข port ให้เป็นเลขเดิมเสมอ และสั่งให้ Flutter เลิกใช้ Temporary Profile ในการบันทึก แล้วไปใช้โฟลเดอร์ที่เรากำหนดขึ้นแทน

# วิธีการทดสอบ (cont.)

- การทดสอบบน **Chrome Browser** **วิธีที่ #1**
- รันผ่าน Web Server วิธีนี้จะใช้ Profile หลักของ Chrome ในการเก็บข้อมูล ทำให้ไม่มีการลบ localStorage ที่

- สมมติว่าต้องการรัน Web Server ด้วย port 50000

1. เปิด Terminal บน VS-Code

2. รันคำสั่ง `flutter run -d web-server --web-port 50000`

หมายเลข Port ที่ต้องการกำหนดให้เหมือนกันทุกครั้งเมื่อรัน  
(หมายเลข Port สามารถระบุได้ไม่เกิน 65535)

- จากนั้นจะปรากฏข้อความค้างอยู่ เช่น
- ให้เปิด Browser ขึ้นมาและไปที่ URL:

**`http://localhost:50000`**

- ทำการทดสอบ



```
Launching lib/main.dart on Web Server in debug mode...  
Waiting for connection from debug service on Web Server...  
lib/main.dart is being served at http://localhost:50000  
The web-server device requires the Dart Debug Chrome extension for  
Chrome or Edge devices for an improved development workflow.
```

Flutter run key commands.

**r** Hot reload. 🔥🔥🔥

**R** Hot restart.

**h** List all available interactive commands.

**d** Detach (terminate "flutter run" but leave application running).

**c** Clear the screen

**q** Quit (terminate the application on the device).

# วิธีการทดสอบ (cont.)

- การทดสอบบน **Chrome Browser** **วิธีที่ #2**
- วิธีการทดสอบโดยกำหนดให้ Flutter เลิกใช้ Temporary Profile ในการบันทึก
- สมมติว่าต้องการรันบน Chrome ด้วย port 50000

1. เปิด Terminal บน VS-Code
2. รันคำสั่ง

```
flutter run -d chrome --web-port 50000 --web-browser-flag="--user-data-dir=.chrome_profile"
```

- จากนั้น Chrome Browser จะปรากฏขึ้นเองหลังจากใช้คำสั่ง สามารถทำการทดสอบได้ตามปกติ

วิธีการทดสอบว่าข้อมูลสถานะการ Login ยังคงเก็บอยู่หรือไม่ ให้ปิด Browser หลังจากแสดงหน้า Homepage โดยไม่ต้องกดปุ่มออกจากระบบ จากนั้นให้เข้าใช้งานอีกครั้ง โดยใช้คำสั่งเดิม ซึ่งควรจะแสดงหน้า Homepage ขึ้นมาได้เลยโดยไม่ผ่านการ Login

# ข้อแนะนำ (เพิ่มเติม)

- การเก็บสถานะการ login หรือ username/password ไว้ในพื้นที่ของ local storage นั้นถึงแม้จะเข้ารหัสไว้ก็ตาม ก็ยังไม่ปลอดภัย แนวทางที่ถูกต้องควรใช้ Session Storage หรือเก็บใน RAM แทน เพราะจะถูกลบทันทีที่ออกจาก App แม้ไม่ได้ใช้คำสั่งลบออกก็ตาม

# ข้อแนะนำ (เพิ่มเติม)

| คุณสมบัติ            | LocalStorage                                                                                                                                                                                   | SessionStorage                                                                                                                                         |
|----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| ระยะเวลาเก็บข้อมูล   | ถาวร (จนกว่าจะลบเอง หรือล้างแคช)                                                                                                                                                               | ชั่วคราว (หายทันทีเมื่อปิด Tab หรือ Browser)                                                                                                           |
| ขอบเขต (Scope)       | แชร์ข้อมูลข้าม Tab/Window ที่มาจาก Domain เดียวกันได้                                                                                                                                          | แยกตาม Tab (ต่อให้เป็น URL เดียวกัน แต่เปิดคนละ Tab ก็ไม่แชร์กัน)                                                                                      |
| ขนาดความจุ           | ประมาณ 5-10 MB (ขึ้นอยู่กับเบราว์เซอร์)                                                                                                                                                        | ประมาณ 5 MB                                                                                                                                            |
| กรณีใช้งานที่เหมาะสม | <ul style="list-style-type: none"><li>- เซฟธีมเว็บ (Dark/Light Mode)</li><li>- การตั้งค่าผู้ใช้ (User Preferences)</li><li>- ข้อมูลที่ไม่ไวต่อความปลอดภัยที่อยากให้คงอยู่หลังปิดเว็บ</li></ul> | <ul style="list-style-type: none"><li>- ข้อมูลฟอร์มกรอกหลายขั้นตอน</li><li>- ตระกร้าสินค้าชั่วคราว</li><li>- State ของการเข้าชมในเซสชันนั้นๆ</li></ul> |

# ข้อแนะนำ (เพิ่มเติม)

- การเก็บข้อมูลชั่วคราวใน RAM หรือ Memory ด้วยวิธีสร้างคลาสขึ้นมาจัดเก็บ จะทำให้เมื่อออกจากโปรแกรม ทุกอย่างก็จะถูกลบออกทั้งหมด แม้ผู้ใช้ไม่ได้สั่งลบ

## เปรียบเทียบ พฤติกรรมบน Browser

สถานการณ์บน Browser

`SessionManager` (In-Memory)

`sessionStorage` จริงของ Web

สลับหน้า/เปิดไป Tab อื่น

ข้อมูลยังคงอยู่

ข้อมูลยังคงอยู่

ปิด Tab หรือปิด Browser

✗ ข้อมูลหายทันที

✗ ข้อมูลหายทันที

กด Refresh หน้าเว็บ (F5)

✗ ข้อมูลหาย (เพราะ RAM ของเว็บถูก Reset ใหม่)

ข้อมูลยังคงอยู่



```

class SessionManager {
  // 1. สร้าง private static instance
  static final SessionManager _instance = SessionManager._internal();

  // 2. Factory constructor เพื่อส่งคืน instance เดียวเสมอ
  factory SessionManager() {
    return _instance;
  }

  // 3. Private named constructor ป้องกันการสร้าง instance ใหม่จากภายนอก
  SessionManager._internal();

  // 4. Map สำหรับเก็บข้อมูล Session ในรูปแบบ Key-Value
  final Map<String, dynamic> _sessionStorage = {};

  /* บันทึกข้อมูลเข้า Session
  void set(String key, dynamic value) {
    _sessionStorage[key] = value;
  }

  /* ดึงข้อมูลจาก Session ตาม Key
  T? get<T>(String key) {
    final value = _sessionStorage[key];
    if (value is T) {
      return value;
    }
    return null;
  }

  /* ตรวจสอบว่ามี Key นี้อยู่ใน Session หรือไม่
  bool containsKey(String key) {
    return _sessionStorage.containsKey(key);
  }

  /* ลบข้อมูลเฉพาะ Key ที่ระบุ
  void remove(String key) {
    _sessionStorage.remove(key);
  }

  /* ล้างข้อมูล Session ทั้งหมด (เช่น ตอน Logout)
  void clear() {
    _sessionStorage.clear();
  }
}

```

Download ตัวอย่างไฟล์ session\_manager.dart [ที่นี่](#)

check login status

```

Future<void> _checkLoginStatus() async {
  // Edit #1: Use SessionManager to get the login status.
  final sessionManager = SessionManager();
  final isLoggedIn = sessionManager.get<bool>('isLoggedIn') ?? false;
  setState(() {
    _isLoggedIn = isLoggedIn;
  });
}

```

login success

```

// Edit #2
final sessionManager = SessionManager();
sessionManager.set('isLoggedIn', true);

```

logout

```

Future<void> _logout(BuildContext context) async {
  // Remove the 'isLoggedIn' key from SessionManager.
  // Edit #3
  final sessionManager = SessionManager();
  sessionManager.remove('isLoggedIn');
}

```

# วิธีการทดสอบ (ทดสอบกับคลาส SessionManager)

- การทดสอบบน Mobile Emulator ให้เข้า App และ Login เข้าสู่หน้า Home จากนั้นให้ปัดออกจาก App และ รันใหม่อีกครั้ง ซึ่งควรจะแสดงหน้า Login เป็นหน้าแรก แม้ไม่ได้ Logout ก็ตาม
- การทดสอบบน Chrome Browser จะเลือกวิธีที่ 1 หรือ 2 ก็ได้
- ในที่นี่จะเลือกใช้วิธีที่ 2 โดยกำหนดให้ Flutter เลิกใช้ Temporary Profile ในการบันทึก
- สมมติว่าต้องการรันบน Chrome ด้วย port 50000
  1. เปิด Terminal บน VS-Code และรันคำสั่ง

```
flutter run -d chrome --web-port 50000 --web-browser-flag="--user-data-dir=.chrome_profile"
```

- เมื่อปรากฏหน้า Login ให้ทำการ Login เพื่อเข้าสู่หน้า Home จากนั้นให้ออกจาก App โดยไม่ต้องทำการ Logout
- ให้เข้า App อีกครั้งด้วยคำสั่งเดิม คราวนี้สถานะการ login ควรถูกลบไปแล้วจาก Memory เมื่อเข้าใหม่ ควรจะปรากฏ หน้า Login เพื่อกรอก Username/Password ใหม่อีกครั้ง

# การบ้าน #5

- ให้นิสิตแก้ไขงานการบ้านชิ้นที่ #4 (Article Reader) โดยให้การตั้งค่า App ภายใน Setting นั้น จะต้องทำการเก็บข้อมูลการ Setting ปัจจุบันไว้ใน Shared Storage ด้วย เพื่อให้ Mode การแสดงผล รวมทั้งชื่อและขนาดของตัวอักษรที่เลือกไว้ ยังคงอยู่และถูกนำกลับมาใช้ตั้งค่าเริ่มต้น เมื่อเปิดใช้ App ในครั้งถัดไป

## การทดสอบ:

- การทำงานในครั้งแรก หากไม่พบข้อมูลตั้งค่าเดิม ให้เริ่มด้วยการใช้ค่า Default คือ Light Mode และขนาดตัวอักษรเป็น Small (14px) (ชื่อ Default Font ให้นิสิตเลือกกำหนดเองตามที่น่ามาใช้)
- เมื่อทำการ Setting โดยเปลี่ยนเป็นค่า เช่น เปลี่ยนเป็น Dark Mode ขนาดตัวอักษรเป็น Large (20px) จะต้องทำการบันทึกการตั้งค่าไว้ใน Shared Storage ด้วยหลังจากปรับเปลี่ยนการแสดงผลแล้ว
- หลังจากออกจากโปรแกรม และเข้าใหม่อีกครั้ง การแสดงผลจะต้องแสดงด้วยการตั้งค่าล่าสุดคือ Dark Mode และขนาดตัวอักษรเป็น Large เป็นต้น

Q/A